
LECTURE NOTES

Lecture 2: Primers on Cryptography

AI510 - Cybersecurity and Innovation

Simon Holm
Fall - 2026



Contents

1	Briefest of Primers on Cryptography	3
2	The setup	3
3	Symmetric encryption	4
3.1	Caesar Cipher	4
3.2	One-Time Pad	4
3.3	AES by NIST	5
3.4	Modes of Operation	6
3.5	Cipher Block Chaining	7
3.6	Hash Functions	7
3.6.1	Cryptographic hashes	7
3.6.2	Hashes for Password Storage	8
4	Asymmetric Encryption	9
5	Hybrid Encryption	9
5.1	Digital signatures	9
5.2	RSA	9
6	Pitfalls in practice	10

1 Briefest of Primers on Cryptography

Definition: Cryptography

The discipline that embodies the principles, means, and methods for providing information security, including confidentiality, data integrity, non- repudiation, and authenticity.

2 The setup

- Confidentiality

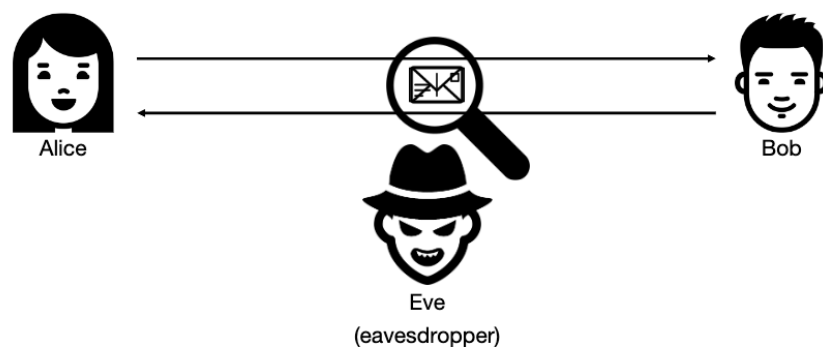


Figure 1: Someone can read your info/message

- Integrity & Authenticity

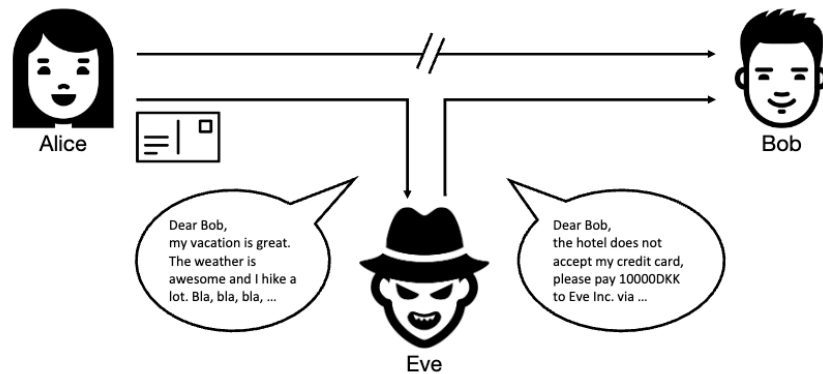


Figure 2: Someone can corrupt your info/message

3 Symmetric encryption

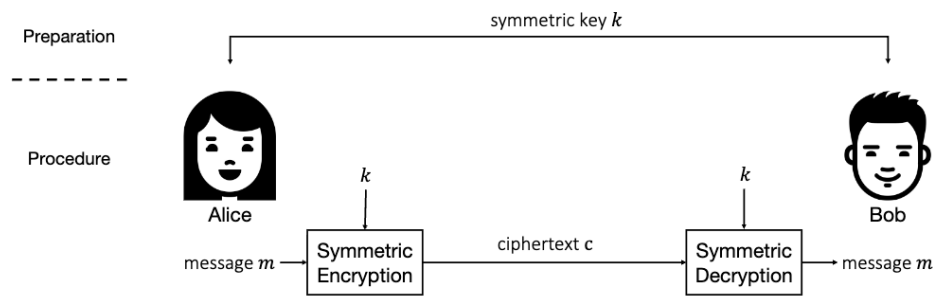


Figure 3: Use keys to encrypt and decrypt messages

3.1 Caesar Cipher

Every character is replaced by another.

Given an interger k , which determines the rotation of encryottion

Example: encryption with $k = 3$

Given the message "Hej"

$$H \rightarrow K, \quad e \rightarrow h, \quad j \rightarrow m$$

So given $k = 3$

$$\text{Hej} \rightarrow \text{Khm}$$

Definition: Kerckhoff's Principle(s)

Has six design rules, most importantly that "The key should be the only secret, not the mechanism"

Secrecy based on the mechanism is called Security by Obscurity which is **bad**.

3.2 One-Time Pad

Message m and key k are added using modular addition.

Example

$$m = 5, \quad k = 15, \quad \text{mod } 17$$

$$m + k \equiv 5 + 15 \equiv 20 = 3 \text{ mod } 17$$

In practice we use bit-wise XOR, modular addition with mod 2

Example

$$m = 1001, k = 1010$$

$$m + k \equiv 1001 \oplus 1010 \equiv 20 = 0011$$

Example: A more complex example

A	B	C	D	E	F	G	H	I	J	K	L	M	N
00000	00001	00010	00011	00100	00101	00110	00111	01000	01001	01010	01011	01100	01101
O	P	Q	R	S	T	U	V	W	X	Y	Z	.	~
00000	00001	00010	00011	00100	00101	00110	00111	01000	01001	01010	01011	01100	01101

Figure 4:

3.3 AES by NIST

Result of an international competition, which encrypts data in blocks by 128 bits. Keys can either be 128, 192 or 256 bits

Consists of 4 operations in 10-14 stages (depending on key length)

- Substitute Bytes: Uses a table called S-Box to perform a byte-by-byte substitution
- Shift Rows: A simple permutation that is performed row by row
- Mix Columns: A substitution that alters each byte in a column as a function of all the bytes in the column
- Add Round Key: A simple bitwise XOR of the current block with a portion of the expanded key

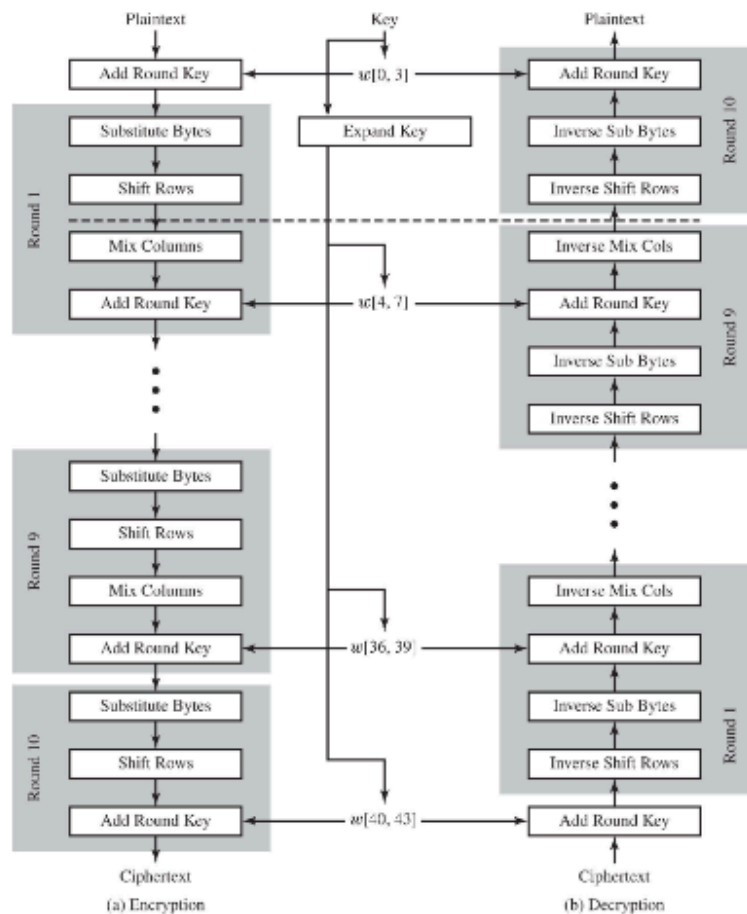


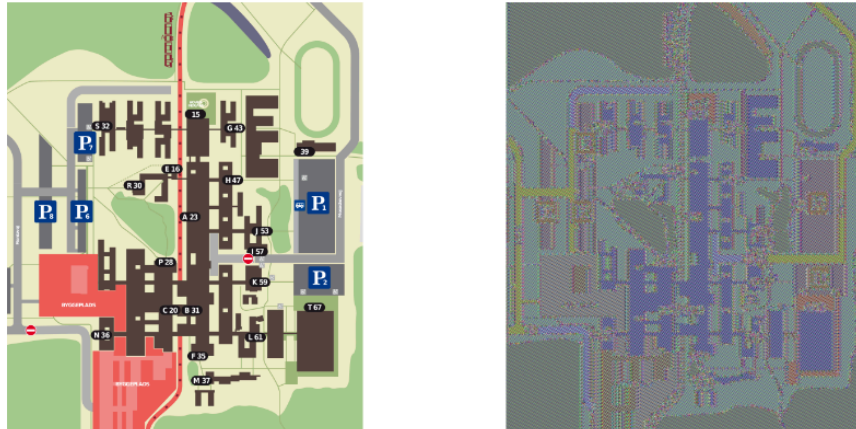
Figure 5: Example
WILL NOT BE IN THE EXAM

3.4 Modes of Operation

These algorithms can be divided into two classes

- Stream-ciphers: Each bit of the data is encrypted and decrypted individually
- Block-ciphers: The data is divided into blocks of fixed length and each block is encrypted individually

Problem: Same plaintexts will result in same ciphertexts keeping the structure of the data intact.



Do not use it!

Figure 6:

3.5 Cipher Block Chaining

Cipher Block Chaining is just one, many likewise alternatives exists (Currently most advanced one is Galois/Counter Mode)

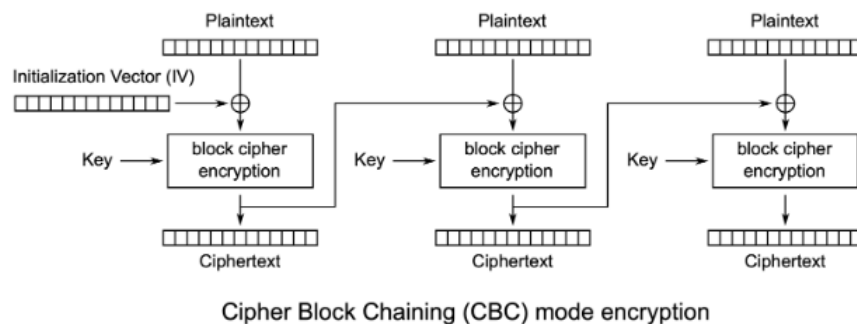


Figure 7: Cipher Block Chaining, relies each encryption on the last. which is great for security but pretty bad for parallelization

3.6 Hash Functions

The goal of all hash functions is to derive values of fixed length from inputs of arbitrary length. These are used for password storage.

3.6.1 Cryptographic hashes

- One-way function

Given output h it is infeasible to find x such that $H(x) = h$

- Collision-resistance

It is infeasible to find any pair (x, y) such that $H(x) = H(y)$

- Efficient

Computation takes little resources

One should (as of 2024) use SHA-3 or SHA-2. Nothing else.

3.6.2 Hashes for Password Storage

More efficient than cryptographic hashes

Originally no dedicated hashes existed, and cryptographic hashes were used mostly. Today we use specialized hashes for password storage. Best practice is “Argon2”.

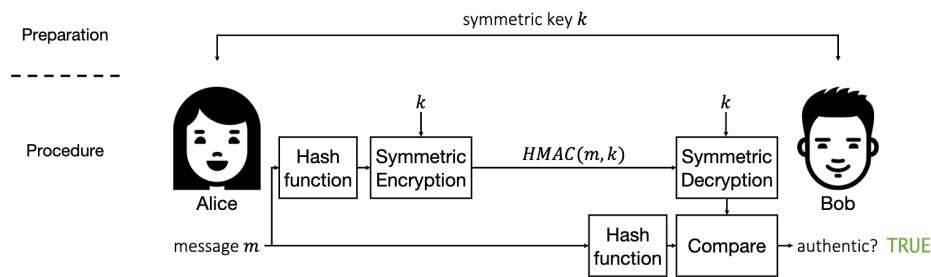


Figure 8:

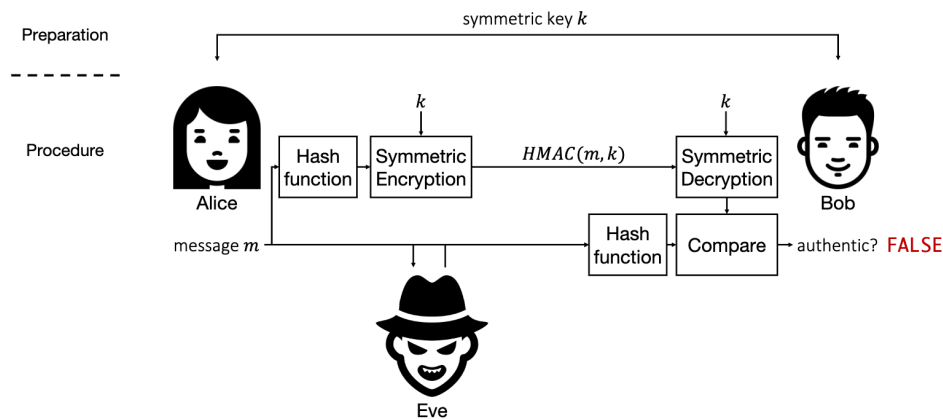


Figure 9:

4 Asymmetric Encryption

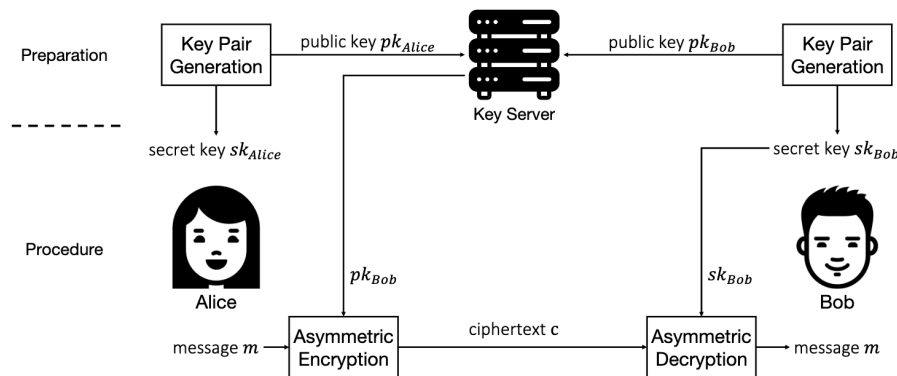


Figure 10:

5 Hybrid Encryption

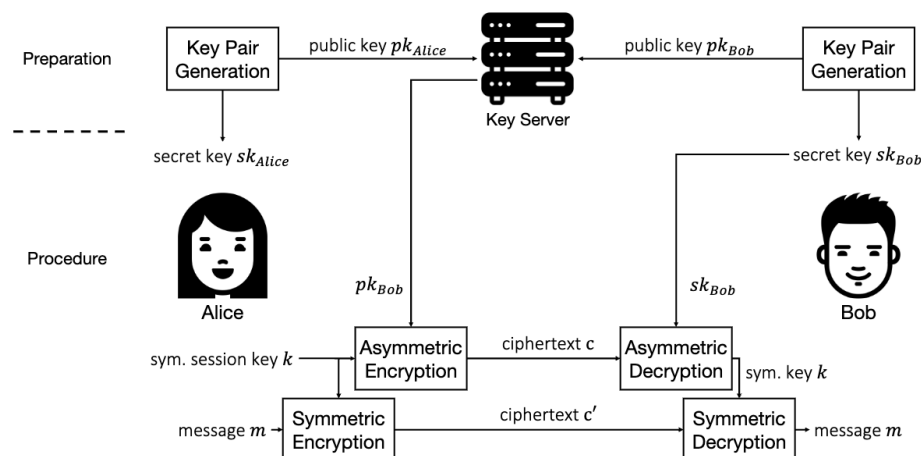


Figure 11:

5.1 Digital signatures

Use either RSA or ECDSA (elliptic curve digital signature) When assuming that the private keys are private, one can create signatures with the private keys (which can be verified with the public key)

5.2 RSA

- Encryption $c \equiv m^e \bmod n$
- Decryption $m \equiv c^d \bmod n \left[\equiv (m^e)^d \bmod n \equiv m^{ed} \right]$

e is the public key, d is the private key

Example: RSA

1. Select $p = 17, q = 11$ (both p and q are prime and $p \neq q$)
2. Calculate $n = p \cdot q = 17 \cdot 11 = 187$
3. Calculate $\phi(n) = (p - 1) \cdot (q - 1) = 160$

4. Select integer e

$$(\gcd(\phi(n), e) = 1; 1 < e < \phi(n))$$

5. Calculate d

$$(d \cdot e \bmod \phi(n) = 1)$$

6 Pitfalls in practice

- Don't implement your own encryption. It will be susceptible to fatal attacks. Examples from the book like (RSA) is deterministic and multiplicative homomorphic
- Side channel attacks. Hardware might leak secrets, (timing or power)